

HOMework 3 SPRING 2025

APPLIED COMPUTER VISION (CMU-AFRICA)

Release Date: Tuesday, February 24th, 2026 CAT

DUE: Sunday March 8th, 2026, 11:59 PM CAT

Augmented Reality with Planar Homographies

In this assignment, you will develop an Augmented Reality (AR) application using planar homographies. It begins with an introduction to the theory behind homographies and their role in perspective transformations. In the coding section, you will identify point correspondences between images to compute a homography matrix, use it to warp images, and ultimately overlay virtual content onto real-world scenes. By the end, you will gain practical experience with key AR techniques applicable in computer vision, robotics, and interactive media.

1. **Collaboration policy:** Students are encouraged to work in groups but each student must submit their own work. If you work as a group, include the names of your collaborators in your write up. Code should NOT be shared or copied. Please DO NOT use external code unless permitted. Plagiarism is strongly prohibited and may lead to failure of this course.
2. **Start early:** This is a much bigger assignment than assignment 1 & 2.
3. **Getting Help:** If you have any questions, please look at Piazza first. Other students may have encountered the same problem, and it may be solved already. If not, post your question on the discussion board. Teaching staff will respond as soon as possible.
4. **Reporting:** Your write-up should mainly consist of three parts, your answers to theory questions, resulting images of each step, and the discussions for experiments. Please note that we DO NOT accept handwritten scans for your write-up in this assignment. Please type your answers to theory questions and discussions for experiments electronically.
5. Please stick to the function prototypes mentioned in the handout. This makes verifying code easier for the TAs.
6. **File paths:** Please make sure that any file paths that you use are relative and not absolute. Not `cv2.imread('/name/Documents/subdirectory/hw2/data/xyz.jpg')` but `cv2.imread('../data/xyz.jpg')`.
7. **Submitting your work:** Create a `<andrew-id>.pdf` and zip file, `<andrew-id>.zip`, composed of your Python implementations (including helper functions), and your implementations & results. Please make sure to remove the `data/` folder, `loadVid.py`, `helper.py`, and any other temporary files you've generated. Your final upload should have the files arranged in this layout:
 - AndrewId.pdf

- AndrewId.zip
 - code/
 - * ar.py
 - * briefRotTest.py
 - * HarryPotterize.py
 - * matchPics.py
 - * planarH.py
 - * classify.py
 - * helper.py
 - * q2_4.py
 - * yourHelperFunctions.py (optional)
 - results/
 - * ar.avi
 - * dataset_grid.png
 - * augmentation_examples.png
 - * confusion_matrix.png

Homographies

Planar Homographies as a Warp

A planar homography is a transformation that maps pixel coordinates from one camera frame to another, assuming that the points being mapped lie on a single plane in the real world. This key assumption allows for a direct correspondence between pixel locations in one view and their counterparts in another view of the same plane. Under this condition, the relationship between the two views can be represented by a homography matrix $\mathbf{H}$, which defines how the points in one image are warped to align with the perspective of the second image.

$$\mathbf{x}_1 \equiv \mathbf{H}\mathbf{x}_2 \quad (1)$$

The $\equiv$ symbol stands for “identical to.” The points $\mathbf{x}_1$ and $\mathbf{x}_2$ are in homogeneous coordinates, which means they have an additional dimension. If $\mathbf{x}_1$ is a 3D vector:

$$\mathbf{x}_1 = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} \quad (2)$$

it represents the 2D point in inhomogeneous or heterogeneous coordinates:

$$\left(\frac{x_i}{z_i}, \frac{y_i}{z_i} \right) \quad (3)$$

This additional dimension is a mathematical convenience to represent transformations (such as translation, rotation, scaling, etc.) in a concise matrix form. The $\equiv$ symbol means that the equation is correct up to a scaling factor.

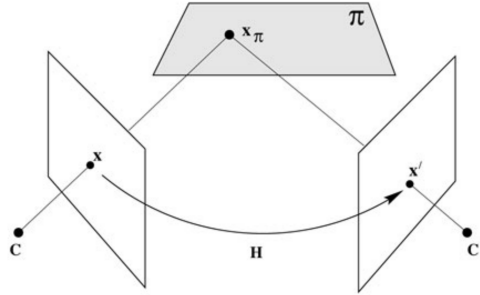


Figure 1: A homography $\mathbf{H}$ links all points $\mathbf{x}_\pi$ lying in plane π between two camera views $\mathbf{x}$ and $\mathbf{x}'$ in cameras C and C' respectively such that $\mathbf{x}' = \mathbf{H}\mathbf{x}$. [From Hartley and Zisserman]

The Direct Linear Transform

A very common problem in projective geometry is often of the form $\mathbf{x} \equiv \mathbf{A}\mathbf{y}$, where $\mathbf{x}$ and $\mathbf{y}$ are known vectors, and $\mathbf{A}$ is a matrix which contains unknowns to be solved. Given matching points in two images, our homography relationship clearly is an instance of such a problem. Note that the equality holds only up to scale (which means that the set of equations are of the form $\mathbf{x} = \lambda\mathbf{H}\mathbf{x}'$), which is why we cannot use an ordinary least squares solution such as what you may have used in the past to solve simultaneous equations. A standard approach to solve these kinds of problems is called the Direct Linear Transform, where we rewrite the equation as proper homogeneous equations which are then solved in the standard least

squares sense. Since this process involves disentangling the structure of the $\mathbf{H}$ matrix, it's a transform of the problem into a set of linear equations, thus giving it its name.

Q1.1 Correspondences

(10 points)

Let $\mathbf{x}_1$ be a set of points in an image and $\mathbf{x}_2$ be the set of corresponding points in an image taken by another camera. Suppose there exists a homography $\mathbf{H}$ such that:

$$\mathbf{x}_1^i \equiv \mathbf{H}\mathbf{x}_2^i \quad (i \in \{1 \dots N\}) \quad (4)$$

where $\mathbf{x}_1^i = [x_1^i \ y_1^i \ 1]^T$ are in homogeneous coordinates, $\mathbf{x}_1^i \in \mathbf{x}_1$ and $\mathbf{H}$ is a 3×3 matrix. For each point pair, this relation can be rewritten as

$$\mathbf{A}_i \mathbf{h} = \mathbf{0} \quad (5)$$

where $\mathbf{h}$ is a column vector reshaped from $\mathbf{H}$, and $\mathbf{A}_i$ is a matrix with elements derived from the points $\mathbf{x}_1^i$ and $\mathbf{x}_2^i$. This can help calculate $\mathbf{H}$ from the given point correspondences.

1. How many degrees of freedom does $\mathbf{h}$ have? (1 points)
2. How many point pairs are required to solve $\mathbf{h}$? (1 points)
3. Derive $\mathbf{A}_i$. (4 points)
4. When solving $\mathbf{A}\mathbf{h} = \mathbf{0}$, in essence you're trying to find the $\mathbf{h}$ that exists in the null space of $\mathbf{A}$. What that means is that there would be some non-trivial solution for $\mathbf{h}$ such that the product $\mathbf{A}\mathbf{h}$ turns out to be $\mathbf{0}$.

What will be a trivial solution for $\mathbf{h}$? Is the matrix $\mathbf{A}$ full rank? Why/Why not? What impact will it have on the singular values? What impact will it have on the singular vectors? (4 points)

Using Matrix Decompositions to Calculate the Homography

A homography $\mathbf{H}$ transforms one set of points (in homogeneous coordinates) to another set of points. In this project, we will obtain the corresponding point coordinates using feature matches and will then need to calculate the homography. You have already derived that $\mathbf{A}\mathbf{x} = \mathbf{0}$ in Question 1. In this section, we will look at how to solve such equations using two approaches, either of which can be used in the subsequent assignment questions.

Eigenvalue Decomposition

One way to solve $\mathbf{A}\mathbf{x} = \mathbf{0}$ is to calculate the eigenvalues and eigenvectors of $\mathbf{A}$. The eigenvector corresponding to 0 is the answer for this. Consider this example:

$$\mathbf{A} = \begin{bmatrix} 3 & 6 & -8 \\ 0 & 0 & 6 \\ 0 & 0 & 2 \end{bmatrix} \quad (6)$$

Using the `numpy.linalg` function `eig`, we get the following eigenvalues and eigenvectors:

$$\mathbf{V} = \begin{bmatrix} 1.0000 & -0.8944 & -0.9535 \\ 0 & 0.4472 & 0.2860 \\ 0 & 0 & 0.0953 \end{bmatrix} \quad (7)$$

$$\mathbf{D} = \begin{bmatrix} 3 & 0 & 2 \end{bmatrix} \quad (8)$$

Here, the columns of $\mathbf{V}$ are the eigenvectors and each corresponding element in $\mathbf{D}$ is its eigenvalue. We notice that there is an eigenvalue of 0. The eigenvector corresponding to this is the solution for the equation $\mathbf{A}\mathbf{x} = \mathbf{0}$.

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 3 & 6 & -8 \\ 0 & 0 & 6 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} -0.8944 \\ 0.4472 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (9)$$

Singular Value Decomposition

The Singular Value Decomposition (SVD) of a matrix $\mathbf{A}$ is expressed as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (10)$$

Here, $\mathbf{U}$ is a matrix of column vectors called the “left singular vectors”. Similarly, $\mathbf{V}$ is called the “right singular vectors”. The matrix $\mathbf{\Sigma}$ is a diagonal matrix. Each diagonal element σ_i is called the “singular value” and these are sorted in order of magnitude. In our case, it is a 9×9 matrix.

- If $\sigma_9 = 0$, the system is exactly-determined, a homography exists and all points fit exactly.
- If $\sigma_9 \geq 0$, the system is over-determined. A homography exists but not all points fit exactly (they fit in the least-squares error sense). This value represents the goodness of fit.
- Usually, you will have at least four correspondences. If not, the system is under-determined. We will not deal with those here.

The columns of $\mathbf{U}$ are eigenvectors of $\mathbf{A}\mathbf{A}^T$. The columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{A}^T\mathbf{A}$. We can use this fact to solve for $\mathbf{h}$ in the equation $\mathbf{A}\mathbf{h} = \mathbf{0}$. Using this knowledge, let us reformulate our problem of solving $\mathbf{A}\mathbf{x} = \mathbf{0}$. We want to minimize the error in solution in the least-squares sense. Ideally, the product $\mathbf{A}\mathbf{h}$ should be $\mathbf{0}$. Thus the sum-squared error can be written as:

$$f(\mathbf{h}) = \frac{1}{2}(\mathbf{A}\mathbf{h} - \mathbf{0})^T(\mathbf{A}\mathbf{h} - \mathbf{0}) \quad (11)$$

$$= \frac{1}{2}(\mathbf{A}\mathbf{h})^T(\mathbf{A}\mathbf{h}) \quad (12)$$

$$= \frac{1}{2}\mathbf{h}^T\mathbf{A}^T\mathbf{A}\mathbf{h} \quad (13)$$

Minimizing this error with respect to $\mathbf{h}$, we get:

$$\frac{d}{d\mathbf{h}}f = \mathbf{0} \quad (14)$$

$$\implies \frac{1}{2}(\mathbf{A}^T\mathbf{A} + (\mathbf{A}^T\mathbf{A})^T)\mathbf{h} = \mathbf{0} \quad (15)$$

$$\mathbf{A}^T\mathbf{A}\mathbf{h} = \mathbf{0} \quad (16)$$

This implies that the value of $\mathbf{h}$ equals the eigenvector corresponding to the zero eigenvalue (or closest to zero in case of noise). Thus, we choose the smallest eigenvalue of $\mathbf{A}^T\mathbf{A}$, which is σ_9 in $\mathbf{\Sigma}$ and the least-squares solution to $\mathbf{A}\mathbf{h} = \mathbf{0}$ is the corresponding eigenvector (in column 9 of the matrix $\mathbf{V}$).

Computing Planar Homographies

Feature Detection and Matching

Before finding the homography between an image pair, we need to find corresponding point pairs between two images. But how do we get these points? One way is to select them manually, which is tedious and inefficient. The CV way is to find interest points in the image pair and automatically match them. In the interest of being able to do cool stuff, we will not reimplement a feature detector or descriptor here, but use python modules. The purpose of an interest point detector (e.g. Harris, SIFT, SURF, etc.) is to find particular salient points in the images around which we extract feature descriptors (e.g. MOPS, etc.). These descriptors try to summarize the content of the image around the feature points in as succinct yet descriptive manner possible (there is often a trade-off between representational and computational complexity for many computer vision tasks; you can have a very high dimensional feature descriptor that would ensure that you get good matches, but computing it could be prohibitively expensive). Matching, then, is a task of trying to find a descriptor in the list of descriptors obtained after computing them on a new image that best matches the current descriptor. This could be something as simple as the Euclidean distance between the two descriptors, or something more complicated, depending on how the descriptor is composed. For the purpose of this exercise, we shall use the widely used FAST detector in concert with the BRIEF descriptor.

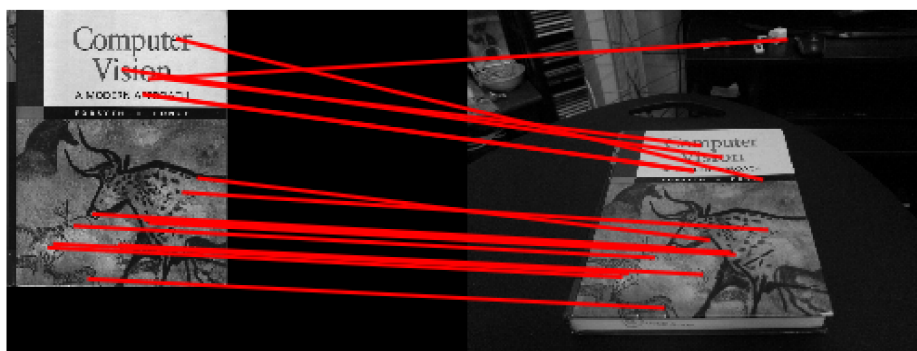


Figure 2: A few matched FAST feature points with the BRIEF descriptor.

Q2.1 FAST Detector

(3 points)

How is the FAST detector different from the Harris corner detector that you've seen in the lectures? (You will probably need to look up the FAST detector online.) Can you comment on its computational performance vis-à-vis the Harris corner detector?

Q2.2 BRIEF Descriptor

(3 points)

How is the BRIEF descriptor different from the filterbanks you've seen in the lectures? Could you use any one of those filter banks as a descriptor?

Q2.3 Matching Methods

(3 points)

The BRIEF descriptor belongs to a category called binary descriptors. In such descriptors the image region corresponding to the detected feature point is represented as a binary string of 1s and 0s. A commonly used metric used for such descriptors is called the Hamming distance. Please search online to learn about Hamming distance and Nearest Neighbor, and describe

how they can be used to match interest points with BRIEF descriptors. What benefits does the Hamming distance have over a more conventional Euclidean distance measure in our setting?

Q2.4 Feature Matching

(6 points)

The number of matches between the two images varies based on the parameter `sigma` used in `corner_detection`, and also on the value `ratio` in `briefMatch`. You should vary these to get the best results. The example shown in Fig. 2 is with `sigma = 0.15` and `ratio = 0.65`.

We provide you with the following helper functions:

```
locs = corner_detection(img, sigma)
desc, locs = computeBrief(img, locs)
matches = briefMatch(desc1, desc2)
plotMatches(im1, im2, matches, locs1, locs2)
```

`locs` is an $N \times 2$ matrix in which each row represents the location (x, y) of a feature point. Please note that the number of valid output feature points can be less than the number of input feature points. `desc` is the corresponding matrix of BRIEF descriptors for the interest points.

Tasks

Please implement a function:

```
matches, locs1, locs2 = matchPics(I1, I2)
```

where `I1` and `I2` are the images you want to match. `locs1` and `locs2` are $N \times 2$ matrices containing the x and y coordinates of the feature points. `matches` is a $p \times 2$ matrix where the first column is indices into features in `I1`, and similarly the second column contains indices related to `I2`. Use the provided helper function `corner_detection` to compute the features, then build descriptors using the provided helper function `computeBrief`, and finally compare them using the provided helper function `briefMatch`.

We have provided a script `q2_4.py` that loads `cv_cover.jpg` and `cv_desk.png` and calls your `matchPics` function to test your implementation. You should use this script to verify your implementation is correct. **Note:** `q2_4.py` is a provided testing script and should **not** be included in your submission.

In your write-up: Use the provided helper function `plotMatches` to visualize your matched points and include the resulting image in your write-up.

Q2.5 BRIEF and Rotations

(8 points)

Let's investigate how BRIEF works with rotations.

Tasks

Write a script `briefRotTest.py` that:

- Takes the `cv_cover.jpg` and matches it to itself rotated [Hint: use `scipy.ndimage.rotate`] in increments of 10 degrees.
- Stores a histogram of the count of matches for each orientation.

- Plots the histogram using `matplotlib.pyplot.bar`.

In your write-up: Include visualizations of the feature matching results at three different orientations. Explain why you think the BRIEF descriptor behaves this way.

Homography Computation

Q2.6 Computing the Homography (13 points)

$\mathbf{x}_1$ and $\mathbf{x}_2$ are $N \times 2$ matrices containing the coordinates (x, y) of point pairs between the two images. $\mathbf{H}_{2 \rightarrow 1}$ should be a 3×3 matrix for the best homography from image 2 to image 1 in the least-square sense. The `numpy.linalg` functions `eig` or `svd` will be useful to get the eigenvectors (see Section 1 of this handout for details).

Tasks

Write a function `computeH` that estimates the planar homography from a set of matched point pairs:

```
H2to1 = computeH(x1, x2)
```

Homography Normalization

Normalization improves numerical stability of the solution and you should always normalize your coordinate data. Normalization has two steps:

1. Translate the mean of the points to the origin.
2. Scale the points so that the largest distance to the origin is $\sqrt{2}$.

This is a linear transformation and can be written as follows:

$$\tilde{\mathbf{x}}_1 = \mathbf{T}_1 \mathbf{x}_1 \quad (17)$$

$$\tilde{\mathbf{x}}_2 = \mathbf{T}_2 \mathbf{x}_2 \quad (18)$$

where $\tilde{\mathbf{x}}_1$ and $\tilde{\mathbf{x}}_2$ are the normalized homogeneous coordinates of $\mathbf{x}_1$ and $\mathbf{x}_2$. $\mathbf{T}_1$ and $\mathbf{T}_2$ are 3×3 matrices.

The homography $\tilde{\mathbf{H}}$ from $\tilde{\mathbf{x}}_2$ to $\tilde{\mathbf{x}}_1$ computed by `computeH` satisfies:

$$\tilde{\mathbf{x}}_1 = \tilde{\mathbf{H}} \tilde{\mathbf{x}}_2 \quad (19)$$

By substituting $\tilde{\mathbf{x}}_1$ and $\tilde{\mathbf{x}}_2$ with $\mathbf{T}_1 \mathbf{x}_1$ and $\mathbf{T}_2 \mathbf{x}_2$, we have:

$$\mathbf{T}_1 \mathbf{x}_1 = \tilde{\mathbf{H}} \mathbf{T}_2 \mathbf{x}_2 \quad (20)$$

$$\mathbf{x}_1 = \mathbf{T}_1^{-1} \tilde{\mathbf{H}} \mathbf{T}_2 \mathbf{x}_2 \quad (21)$$

Q2.7 Homography with Normalization (6 points)

This function should normalize the coordinates in $\mathbf{x}_1$ and $\mathbf{x}_2$ and call `computeH(x1, x2)` as described above.

Tasks

Implement the function `computeH_norm`:

```
H2to1 = computeH_norm(x1, x2)
```

RANSAC

The RANSAC algorithm can generally fit any model to noisy data. You will implement it for (planar) homographies between images. Remember that 4 point-pairs are required at a minimum to compute a homography.

Q2.8 RANSAC for Computing a Homography (20 points)

$\mathbf{x}_1$ and $\mathbf{x}_2$ are $N \times 2$ matrices containing the matched points. `inliers` is a vector of length N with a 1 at those matches that are part of the consensus set, and 0 elsewhere. Use `computeH_norm` to compute the homography.

Tasks

Write a function:

```
bestH2to1, inliers = computeH_ransac(x1, x2)
```

where `bestH2to1` should be the homography $\mathbf{H}$ with most inliers found during RANSAC. $\mathbf{H}$ will be a homography such that if $\mathbf{x}_2$ is a point in $\mathbf{x}_2$ and $\mathbf{x}_1$ is a corresponding point in $\mathbf{x}_1$, then $\mathbf{x}_1 \equiv \mathbf{H}\mathbf{x}_2$.

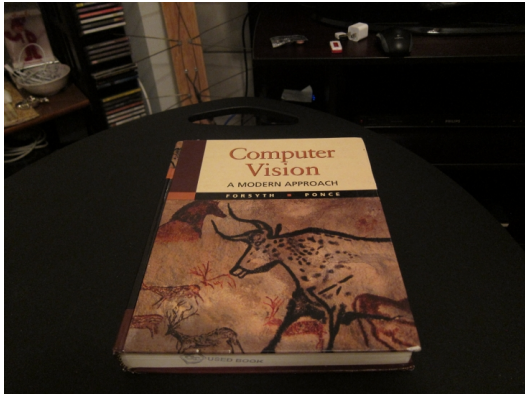


Figure 3: Text book

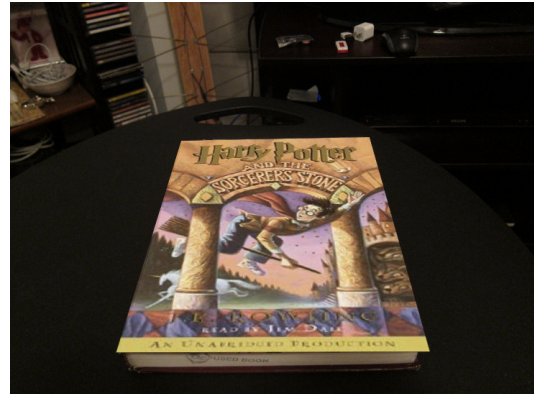


Figure 4: HarryPotterized Text book

Automated Homography Estimation and Warping

Q2.9 Putting it Together (8 points)

Tasks

Write a script `HarryPotterize.py` that:

1. Reads `cv_cover.jpg`, `cv_desk.png`, and `hp_cover.jpg`.
2. Computes a homography automatically using `MatchPics` and `computeH_ransac`.

3. Uses the computed homography to warp `hp_cover.jpg` to the dimensions of the `cv_desk.png` image using the skimage function `skimage.transform.warp` or OpenCV function `cv2.warpPerspective`
4. At this point you should notice that although the image is being warped to the correct location, it is not filling up the same space as the book. Why do you think this is happening? How would you modify `hp_cover.jpg` to fix this issue?
5. Implement the function:

```
composite_img = compositeH( H2to1, template, img )
```

to now compose this warped image with the desk image as in Figure 4.

In your write-up: Show us your final image, `composite_img`, generated by your script `HarryPotterize.py`. Also, discuss the questions raised in item #4.

Creating your Augmented Reality Application

Q3.1 Incorporating Video

(18 points)

Now with the code you have, you're able to create your own Augmented Reality application. What you're going to do is HarryPotterize the video `ar_source.mov` onto the video `book.mov`. More specifically, you're going to track the computer vision text book in each frame of `book.mov`, and overlay each frame of `ar_source.mov` onto the book in `book.mov`.

Note that the book and the videos we have provided have very different aspect ratios (the ratio of the image width to the image height). You must crop each frame to fit onto the book cover. You should crop each frame such that only its central region is used in the final output. See Figure 6 for an example.

Also, the video `book.mov` only has translation of objects. If you want to account for rotation of objects, scaling, etc, you would have to pick a better feature point representation (like ORB).

Tasks

Please write a script `ar.py` to implement this AR application and save your result video as `ar.avi` in the `result/` directory. You may use the function `loadVid` that we provide to load the videos. You'll be given full credits if you can put the video together correctly. See Figure 5 for an example frame of what the final video should look like.



Figure 5: Rendering video on a moving target

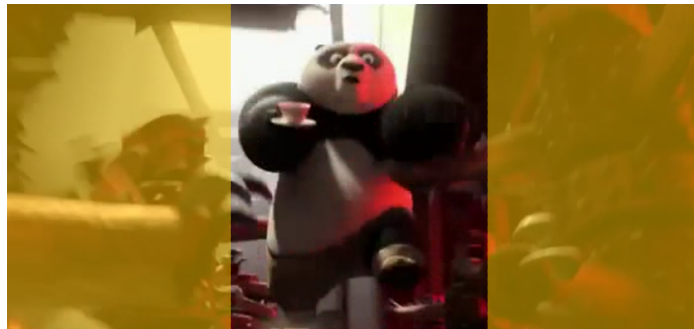


Figure 6: Crop out the yellow regions of each frame to match the aspect ratio of the book

Image Classification with Handcrafted Features

Q4.1 Binary Image Classification

(23 points)

So far in this assignment you have used FAST and BRIEF to *match* images. In this question you will reuse the same feature extraction pipeline to *classify* images, demonstrating how the same building blocks underpin a full Traditional Machine Learning system.

The pipeline you will implement is shown in Figure 7. It follows the classical approach of separating handcrafted feature extraction from the learning stage, in contrast to deep learning where both stages are learned end-to-end.

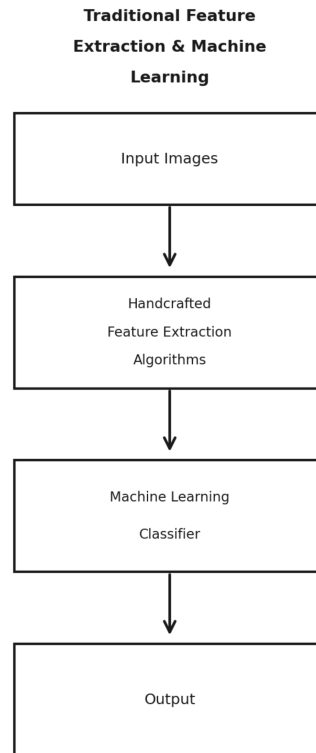


Figure 7: The Traditional Feature Extraction & Machine Learning pipeline you will implement in this question.

Part A — Dataset Setup

(2 points)

We provide you with a binary image classification dataset: **Face vs No-Face**. The dataset contains images with human faces and images without faces. All images have been preprocessed to the same dimensions.

The dataset is included in the assignment assets you downloaded. Verify it is in your `data/` directory with the following structure:

```

data/
| classify/
|   face/           ← images containing faces
|     img_001.jpg
|     img_002.jpg
|     ...
|   no_face/        ← images without faces
|     img_001.jpg
|     img_002.jpg
|     ...
  
```

Write-up — Part A:

1. Include the **dataset grid** generated by `show_dataset_grid()` — saved to `results/dataset_grid.png` — showing at least **4 images per class**.
2. Include the **file path list** printed to the console, confirming all images were loaded correctly.

Part B — Data Augmentation

(5 points)

Because the dataset is relatively small, you will use **data augmentation** to artificially expand it before training. Augmentation applies controlled transformations to existing images, producing new training examples that simulate additional real-world variation.

Task: Choose **exactly 2 factors of variation** to simulate through augmentation. Table 1 lists common examples. Your chosen factors should be relevant to the Face vs No-Face classification task.

Category	Factor	Example transformation
Photometric	Brightness	Add/subtract pixel intensity
Photometric	Contrast	Histogram equalization, gamma correction
Geometric	Rotation	Rotate image by small angle ($\pm 15^\circ$)
Geometric	Flip	Horizontal or vertical flip
Geometric	Scale	Zoom in/out slightly
Sensor	Blur	Gaussian or motion blur
Sensor	Noise	Add random noise

Table 1: Examples of factors of variation you can simulate through augmentation.

Set `AUGMENTATION_FACTOR_1` and `AUGMENTATION_FACTOR_2` at the top of `classify.py` to the names of your two chosen factors (e.g. 'Brightness', 'Rotation').

Complete the `augment_images(images, labels)` function in `classify.py`. For each original image, produce **one augmented copy per factor** (2 extra copies per image, tripling the dataset size). Useful OpenCV functions are listed in the code comments.

Write-up — Part B:

1. State your two augmentation factors and explain in **2–3 sentences each**:
 - Why this factor is relevant to real-world variation in the Face vs No-Face task.
 - Which OpenCV transformation you used to simulate it.
 - What range of parameters you chose (e.g., brightness shift of ± 30 pixels, rotation of $\pm 15^\circ$).
2. Answer this question in **2–3 sentences**: *Name one factor of variation that would **hurt** BRIEF descriptor matching if used as augmentation, and explain why.*
3. Include the augmentation visualization grid saved to `results/augmentation_examples.png` (generated by `show_augmentation_examples()`), showing one original and its two augmented versions for each class. Column headings must show your factor names.

Part C — Conceptual Questions in Code

(4 points)

Inside `classify.py` you will find **conceptual questions** embedded as comment blocks in the functions `extract_features` and `train_classifier`. You must **answer each question directly in the comment block** before writing any code. The questions are:

1. Why do we resize all images to `TARGET_SIZE` before extracting FAST keypoints? What could go wrong if images had different sizes?

2. Why can we not pass a variable-size descriptor matrix ($K \times D$) directly to a classifier, and how does mean pooling solve this?
3. What information is lost when you mean-pool K descriptors into one vector, and how might this limit accuracy?
4. How does your chosen classifier make a prediction given a new feature vector, and why is it appropriate for this small dataset?

Part D — Implementation

(8 points)

Task D1 — Image Shape

(1 point)

At the top of `classify.py`, set `TARGET_SIZE` to your chosen (`height`, `width`) in pixels. This should match the dimensions of the provided dataset images. Justify your choice in the write-up.

Task D2 — Feature Extraction

(3 points)

Complete `extract_features(images, sigma)`:

1. Detect keypoints using `corner_detection(img, sigma)`.
2. Compute BRIEF descriptors using `computeBrief(img, locs)`.
3. Aggregate into a fixed-length vector via mean pooling (Equation 22).
4. Handle the edge case of zero keypoints (zero vector of dim D).
5. Return feature matrix of shape $(N \times D)$.

$$\mathbf{f}_{\text{image}} = \frac{1}{K} \sum_{k=1}^K \mathbf{d}_k \in \mathbb{R}^D \quad (22)$$

Figure 8 illustrates how the variable-size descriptor matrix is compressed into a single fixed-length feature vector via mean pooling.

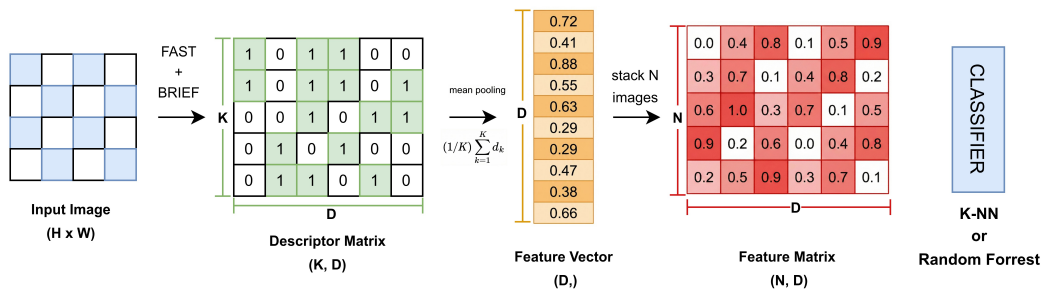


Figure 8: Mean pooling aggregates a variable number of BRIEF descriptors per image into one fixed-length feature vector.

Task D3 — Classifier

(2 points)

Complete `train_classifier(X_train, y_train)`.

Constraint: You may only use one of the following two classifiers. **SVM / SVC is not permitted.**

- k-Nearest Neighbours — `sklearn.neighbors.KNeighborsClassifier`
- Random Forest — `sklearn.ensemble.RandomForestClassifier`

Task D4 — Pipeline Documentation

After running your full pipeline, document the following values in your write-up under Part D (you may present this as a table or list):

- `TARGET_SIZE` ($H \times W$) in pixels
- Augmentation Factor 1 and Factor 2 names
- Total images before augmentation
- Total images after augmentation
- Descriptor dimension D (should be 256)
- Feature matrix shape (N, D)
- Train / Test split sizes
- Classifier used (k-NN or Random Forest)
- Key hyperparameter value (e.g., k for k-NN, `n_estimators` for RF)

Task D5 — Test on Your Own Image

(2 points)

Take a photo using your phone or laptop camera and test your trained classifier on it. The photo can be either:

- A photo of your face (should predict `face`)
- A photo of an object or scene without faces (should predict `no_face`)

Write a function `predict_single_image(image_path, classifier)` that:

1. Loads the image
2. Resizes it to `TARGET_SIZE`
3. Converts to grayscale
4. Extracts features using your `extract_features` function
5. Returns the classifier's prediction and confidence

Write-up — Part D:

1. Include your test image in the write-up.
2. Report the classifier's prediction (`face` or `no_face`).
3. State whether the prediction was correct.
4. If the prediction was incorrect, explain in **2–3 sentences** why you think the classifier failed on this particular image.

Part E — Results & Accuracy Explanation

(4 points)

Run the full pipeline and include the following in your write-up.

Task E1 — Classification Statistics

Include all statistics computed by `evaluate()`: Accuracy, Precision, Recall, F1 Score, and the Confusion Matrix plot saved to `results/confusion_matrix.png`.

Task E2 — Accuracy Explanation

Do not just report the number. You must **explain what it means** by answering all four questions below in your write-up:

1. **Plain-language interpretation.** Restate your accuracy as a sentence, for example: *“My classifier correctly labelled __ out of __ test images, making __ mistake(s).”*
2. **Confusion matrix analysis.** Which class was harder to classify correctly and how can you tell from the confusion matrix? Identify **at least one specific misclassified image**, show it, and explain why you think FAST + BRIEF features caused the error.
3. **Impact of your variation factors.** Did the 2 factors of variation you simulated through augmentation (Part B) help the classifier generalize better? Use your accuracy and confusion matrix to support your answer.
4. **Impact of augmentation.** Explain how your data augmentation affected the result. Re-run the pipeline with augmentation disabled (comment out the `augment_images` call in `main()`) and compare the two accuracy values. Was the difference what you expected? Why or why not?

Submission Checklist

Your submission must include:

- `classify.py` with all TODOs implemented and all conceptual questions answered in the comment blocks.
- `results/dataset_grid.png`
- `results/augmentation_examples.png`
- `results/confusion_matrix.png`
- In your PDF write-up: all items listed in Parts A–E above, including your test image and prediction results from Task D5.